

OBSERVATION TASK – 5

Course: Full Stack Web Development

Course Code: 23CM4121

Experiment No: 5-4

Student Name: Reddy Mohini

Roll No.: A24126552110

Branch: CSM

1. TITLE

Document Object Model (DOM) and Dynamic Modification of HTML Elements using JavaScript

2. AIM

To explain the Document Object Model (DOM) and to demonstrate how JavaScript can dynamically change the content, style, and attributes of an HTML element.

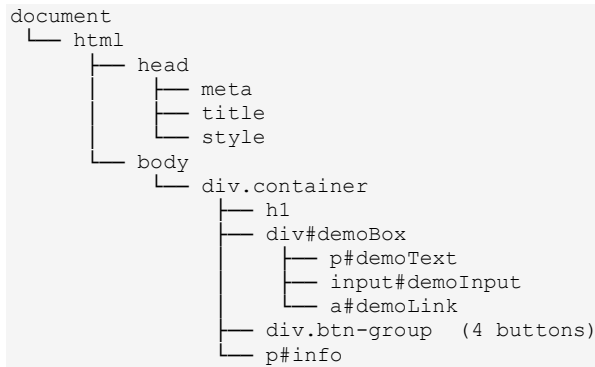
3. OBJECTIVE

- To understand the structure and purpose of the Document Object Model (DOM).
- To select HTML elements using JavaScript DOM methods.
- To change the content of an element using `textContent` / `innerHTML`.
- To change the style of an element using the `style` property.
- To read, modify, and remove attributes using `getAttribute()`, `setAttribute()`, and `removeAttribute()`.
- To use event handling so that changes occur on user interaction.

4. THEORY

What is the DOM? The Document Object Model (DOM) is a programming interface provided by the browser for HTML documents. When a web page loads, the browser reads the HTML and converts it into a tree-like structure of objects called nodes. JavaScript can access this tree and use it to read, add, modify, or delete any part of the page without reloading it.

DOM Tree Structure. Every part of the document is a node: the document itself, each HTML element, each attribute, and each piece of text. Elements are arranged in a parent–child hierarchy, as shown below for the program in this experiment:



Types of nodes: Document node (the root), Element nodes (HTML tags), Attribute nodes (id, class, href, etc.), and Text nodes (the text inside elements).

Selecting elements. Before an element can be changed, it must be selected using methods such as `getElementById()`, `getElementsByClassName()`, `getElementsByTagName()`, `querySelector()`, and `querySelectorAll()`.

Changing an element dynamically. Once an element is selected, JavaScript can modify it in three main ways:

Change	Property / Method	Example
Content	<code>textContent</code> , <code>innerHTML</code>	<code>demoText.textContent = "New text";</code>
Style	<code>element.style.property</code>	<code>demoText.style.color = "blue";</code>
Attributes	<code>setAttribute()</code> , <code>getAttribute()</code> , <code>removeAttribute()</code>	<code>demoLink.setAttribute("href", "url");</code>

Event handling. Events such as clicks are handled using `addEventListener()`, which runs a function whenever the user interacts with an element. Here, button clicks trigger the DOM changes.

Flow of application: User Click → Event Handling → DOM Selection → Modify Content / Style / Attributes → Updated Web Page

5. ALGORITHM / PROCEDURE

1. Create an HTML file with a target element (paragraph, input box, and link) inside a container.
2. Add four buttons: Change Content, Change Style, Change Attributes, and Reset.
3. Select all required elements using `document.getElementById()`.
4. Attach a click event listener to each button using `addEventListener()`.
5. On “Change Content”, update the paragraph using the `textContent` property.
6. On “Change Style”, modify the background color, border, text color, font size, and font weight using the `style` property.

7. On “Change Attributes”, change the placeholder, href, and title attributes using `setAttribute()`.
8. On “Reset”, restore the original content, remove inline styles using `removeAttribute("style")`, and restore original attributes.
9. Display a status message after every action.
10. Test the application by clicking each button and verifying the changes.

6. PROGRAM

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>DOM Manipulation Demo</title>
<style>
  /* Page styling */
  body {
    font-family: Arial, sans-serif;
    background-color: #f2f2f2;
    margin: 0;
    padding: 40px;
  }

  /* Main container */
  .container {
    width: 550px;
    margin: auto;
    background-color: white;
    padding: 25px;
    border-radius: 10px;
    box-shadow: 0 0 10px gray;
  }

  /* Heading */
  h1 {
    text-align: center;
  }

  /* Demo box (element whose content, style and attributes change) */
  #demoBox {
    padding: 20px;
    border: 2px solid #ccc;
    border-radius: 8px;
    background-color: #f8f8f8;
    text-align: center;
  }
```

```

}

#demoText {
    font-size: 18px;
    color: black;
}

#demoInput {
    width: 80%;
    padding: 10px;
    border: 1px solid #ccc;
    border-radius: 5px;
    font-size: 16px;
}

#demoLink {
    display: block;
    margin-top: 15px;
}

/* Buttons */
.btn-group {
    text-align: center;
    margin-top: 20px;
}

button {
    padding: 10px 14px;
    margin: 5px;
    border: none;
    border-radius: 5px;
    color: white;
    cursor: pointer;
}

#contentBtn { background-color: blue; }
#styleBtn   { background-color: green; }
#attrBtn    { background-color: darkorange; }
#resetBtn   { background-color: red; }

/* Status message */
#info {
    text-align: center;

```

```

        color: gray;
        margin-top: 15px;
        font-size: 14px;
    }
</style>
</head>
<body>
    <div class="container">
        <h1>DOM Manipulation Demo</h1>

        <!-- Element to be manipulated -->
        <div id="demoBox">
            <p id="demoText">This is the original paragraph text.</p>
            <input type="text" id="demoInput" placeholder="Original placeholder">
            <a id="demoLink" href="https://www.google.com" title="Go to Google">Visit
Google</a>
        </div>

        <!-- Control buttons -->
        <div class="btn-group">
            <button id="contentBtn">Change Content</button>
            <button id="styleBtn">Change Style</button>
            <button id="attrBtn">Change Attributes</button>
            <button id="resetBtn">Reset</button>
        </div>

        <!-- Status message -->
        <p id="info">Click a button to modify the element.</p>
    </div>

    <script>
        // Select the required HTML elements
        const demoBox = document.getElementById("demoBox");
        const demoText = document.getElementById("demoText");
        const demoInput = document.getElementById("demoInput");
        const demoLink = document.getElementById("demoLink");
        const info = document.getElementById("info");

        // 1. Change CONTENT
        document.getElementById("contentBtn").addEventListener("click", function() {
            demoText.textContent = "The content of this paragraph was changed using
JavaScript!";
            info.textContent = "Content changed using the textContent property.";
        });

        // 2. Change STYLE
        document.getElementById("styleBtn").addEventListener("click", function() {
            demoBox.style.backgroundColor = "#ffe9a8";
            demoBox.style.border = "3px dashed green";
            demoText.style.color = "darkblue";
            demoText.style.fontSize = "24px";
            demoText.style.fontWeight = "bold";
            info.textContent = "Style changed using the style property.";
        });

        // 3. Change ATTRIBUTES
        document.getElementById("attrBtn").addEventListener("click", function() {
            demoInput.setAttribute("placeholder", "Placeholder changed by JavaScript");
            demoLink.setAttribute("href", "https://www.wikipedia.org");
            demoLink.setAttribute("title", "Go to Wikipedia");
            demoLink.textContent = "Visit Wikipedia";
            info.textContent = "Attributes changed. New href = " +
demoLink.getAttribute("href");
        });

        // 4. RESET everything to the original state
        document.getElementById("resetBtn").addEventListener("click", function() {
            demoText.textContent = "This is the original paragraph text.";
            demoBox.removeAttribute("style");
            demoText.removeAttribute("style");
            demoInput.setAttribute("placeholder", "Original placeholder");
            demoLink.setAttribute("href", "https://www.google.com");

```

```

        demoLink.setAttribute("title", "Go to Google");
        demoLink.textContent = "Visit Google";
        info.textContent = "Element reset to its original state.";
    });
</script>
</body>
</html>

```

7. CODE EXPLANATION

Code	Explanation
<code>document.getElementById()</code>	Selects an HTML element using its id and returns it as a DOM object.
<code>addEventListener("click")</code>	Runs a function when the user clicks a button.
<code>element.textContent</code>	Changes the text content of an element.
<code>element.style.property</code>	Changes CSS properties (color, font size, background, border) of an element directly.
<code>setAttribute(name, value)</code>	Changes the value of an existing attribute (or adds a new one), e.g., placeholder, href, title.
<code>getAttribute(name)</code>	Reads the current value of an attribute.
<code>removeAttribute("style")</code>	Removes an attribute; here it clears all inline styles applied by JavaScript.

8. TEST CASES AND EXECUTION DATA

Test Case	Input	Expected Result	Actual Result	Status
TC-01	Open the page	Original text, placeholder, and link are displayed	Works correctly	Pass
TC-02	Click “Change Content”	Paragraph text changes to the new message	Works correctly	Pass
TC-03	Click “Change Style”	Box background, border, and text color/size/weight change	Works correctly	Pass
TC-04	Click “Change Attributes”	Placeholder, link href, and title change	Works correctly	Pass
TC-05	Click “Reset”	All elements return to their.	Works correctly	Pass

9. OUTPUT

DOM Manipulation Demo

This is the original paragraph text.

[Visit Google](#)

Change Content

Change Style

Change Attributes

Reset

Click a button to modify the element.

DOM Manipulation Demo

This is the original paragraph text.

[Visit Google](#)

Change Content

Change Style

Change Attributes

Reset

Style changed using the style property

DOM Manipulation Demo

The content of this paragraph was changed
using JavaScript!

MAMATHA

[Visit Wikipedia](https://www.wikipedia.org)

Change Content

Change Style

Change Attributes

Reset

Attributes changed. New href = <https://www.wikipedia.org>

- Initial page (before any button is clicked)
- After clicking “Change Content”
- After clicking “Change Style”
- After clicking “Change Attributes”
- After clicking “Reset”

10. RESULT

Thus, the Document Object Model (DOM) was studied and a JavaScript program was successfully developed to dynamically change the content, style, and attributes of an HTML element. The application demonstrates DOM selection, DOM manipulation, and event handling.